

Stack $\rightarrow$ int size = 5; $\rightarrow$ Stack size.
 int[] Stack = new int[size]; $\rightarrow$ Stack created.
 int top = -1; $\rightarrow$ initially

4	50
3	40
2	30
1	20
0	10

top = -1;

0
1
2
3
4
size 5

(1) push: $\rightarrow$

top++;

Stack[top] = ele;

(2) Full $\rightarrow$

~~if (top == size - 1)~~

if (top == size - 1)

Stack is Full;

(3) Empty $\rightarrow$

if (top == -1)

Stack is empty;

(4) Display $\rightarrow$ or traverse $\rightarrow$

for (i = top; i >= 0; i--) {
 print(Stack[i]);
}

(5) Peek $\rightarrow$ (top element)

print(Stack[top]);

(6) Pop $\rightarrow$

(delete top element)

print(Stack[top]);

top--;

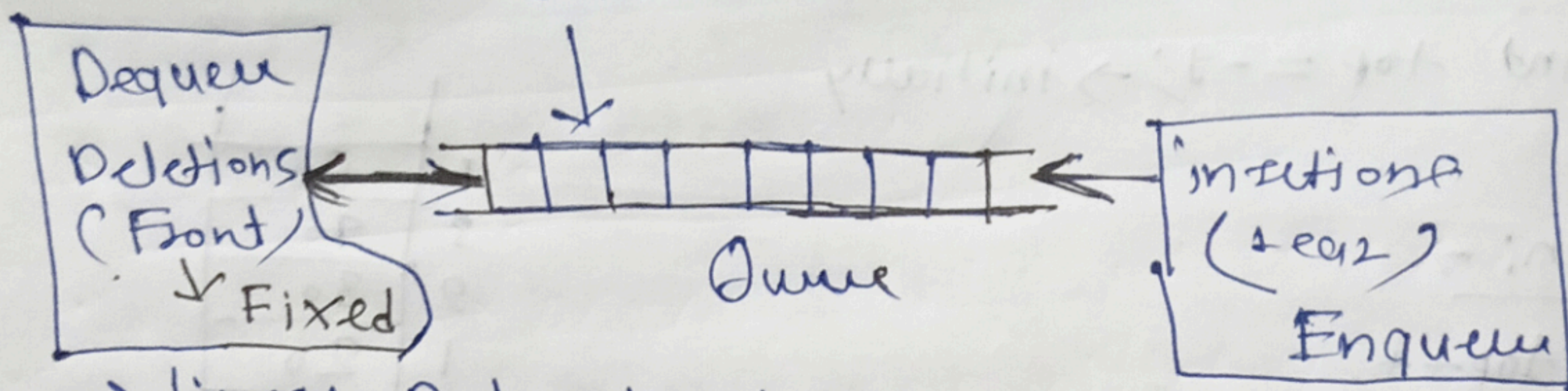
~~it is really~~

$\rightarrow$ it is really remove from memory it is impossible.

$\rightarrow$ replacement if possible.

* Queue $\rightarrow$ ADT

Counter (Fixed location)



$\rightarrow$ Linear Data structure.

$\rightarrow$ (FIFO)

$\rightarrow$ In linear Queue, Front is fixed.

$\rightarrow$ create(): create with specified size

$\rightarrow$ insert():

$\rightarrow$ delete():

$\rightarrow$ traverse():

$\rightarrow$ isEmpty():

$\rightarrow$ isFull():

$\rightarrow$ peek():

```
int size = 5;
```

```
int[] queue = new int[size];
```

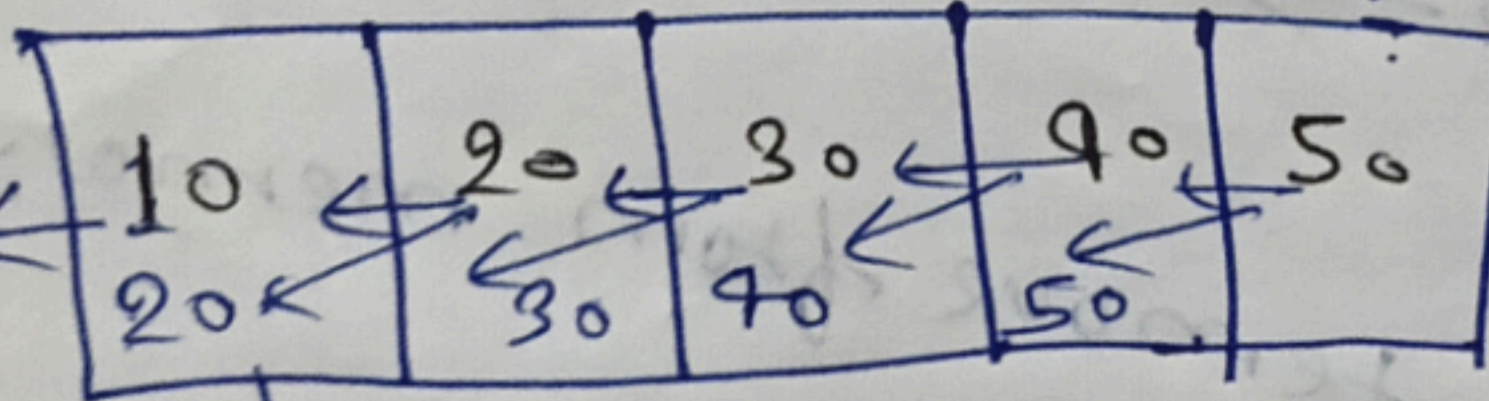
```
int size = 5;
```

```
int[] queue = new int[size];
```

```
int front = 0, rear = -1;
```

front (fixed)

0 $\rightarrow$ 1 $\rightarrow$ 2 $\rightarrow$ 3 $\rightarrow$ 4



Delete front++

(front = 0)

(rear = -1)

0
1
2
3
4

① Insert element →

```
rear++;  
queue[rear] = ele;
```

~~queue[rear]~~

```
queue[++rear] = ele;
```

② Full →

```
if (rear == size - 1)
```

③ Empty →

```
if (rear == -1)
```

④ Traversal →

```
for (int i = front; i <= rear; i++) {  
    print(queue[i]);  
}
```

↘

⑤ Peek → element →

```
print(queue[front]);
```

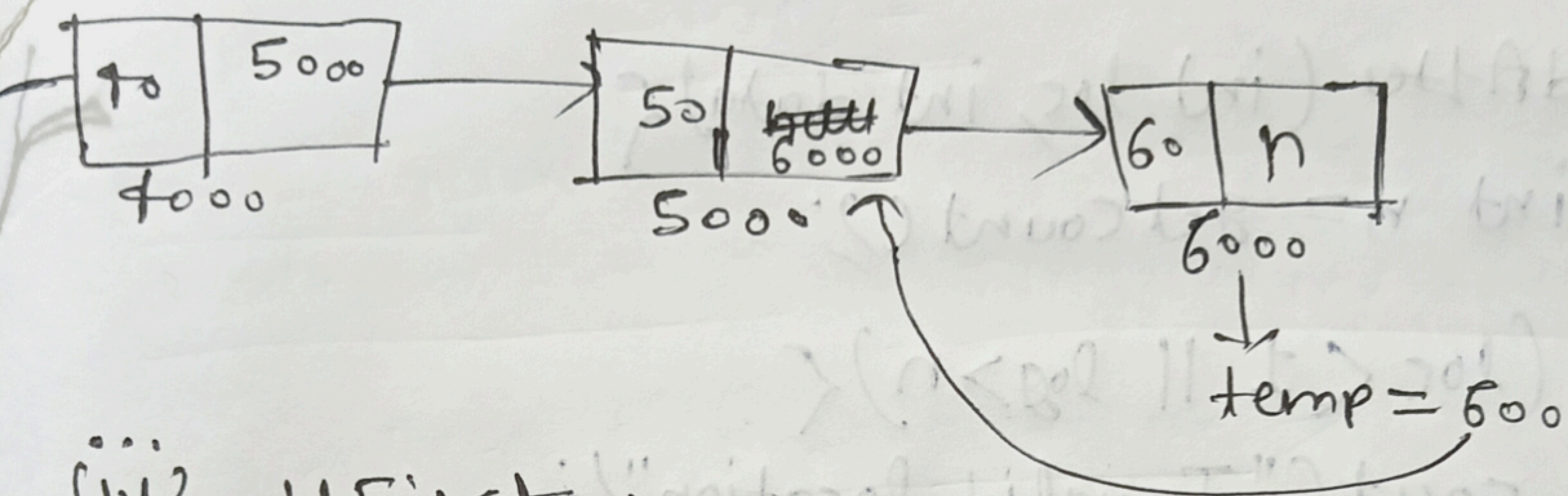
⑥ Delete → element →

```
print("Del: " + queue[front]);
```

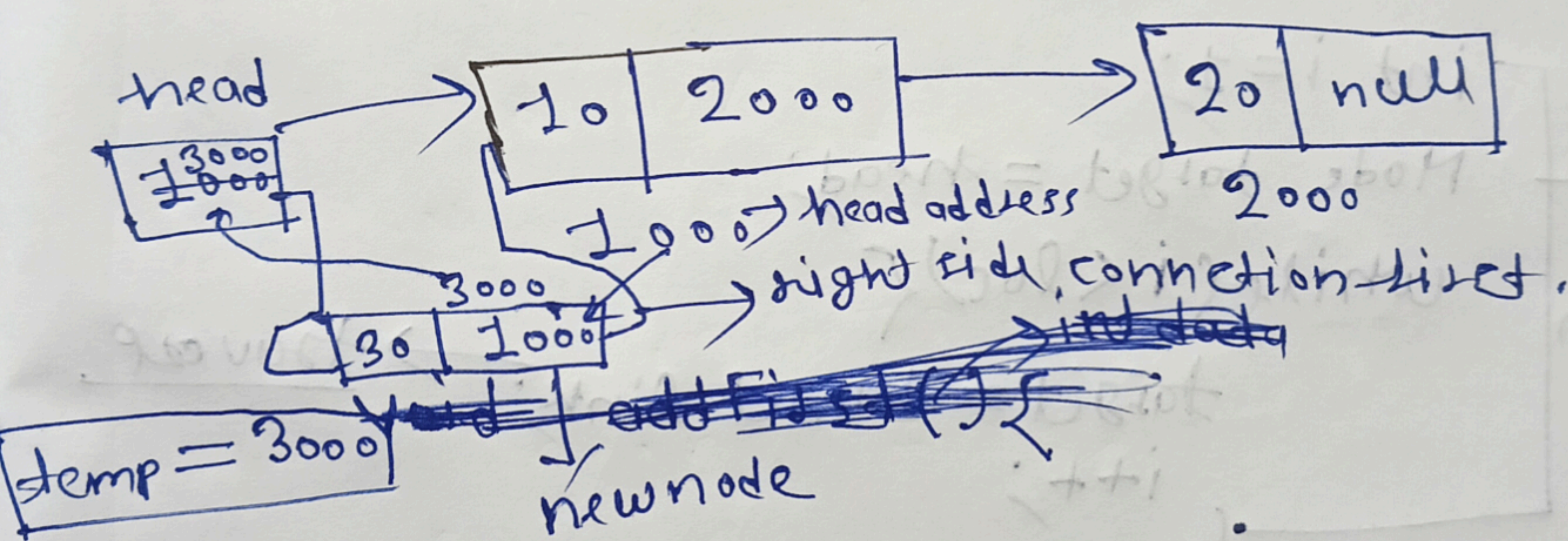
```
for (int i = 0; i <= rear; i++) {
```

```
    queue[i-1] = queue[i];
```

```
    rear--;
```



(iv) add First →



→ (iv) void addFirst(int data) {
 new node ← Node temp = new Node(data);
 temp.link = head; → right side.
 head = temp; → left side.
 }

→ void addAtter (int loc, int data) {
 int n = getCount();
 if (loc < 1 || loc > n) {
 cout < "Invalid location";
 return;
 }
 }

check
location

int i = 1;
 Node target = head;
 while (i < loc) {
 target = target->link;
 i++;
 }
 Node temp = new Node (data);
 temp->link = target->link;
 target->link = temp;

reach
location

node
created

connect
node

traverse

(vi)

→ void delete-first () {
 if (head == null) {
 cout < "Empty list";
 return;
 }
 }

else {

Node target = head;
 head = head->link;
 target->link = null;